

Semi-Decentralized Multi-Spacecraft Collision Avoidance

under Communication Constraints

POMDP Formulation & System Architecture

Grace Ra Kim, Mahdi Al-Husseini, Duncan Eddy, Mykel J. Kochenderfer

May 5, 2026

Contents

1 Problem Overview

Two active spacecraft are on a conjunction trajectory in LEO. Each spacecraft is controlled by a separate ground operator who receives state updates and conjunction data only during intermittent ground-station contact windows. Operators must plan impulsive maneuvers under partial observability and limited coordination.

We model this as a **Semi-Decentralized POMDP (SDec-POMDP)** solved with Recursive Small-Step Semi-Decentralized A* (RS-SDA*). Ground-station contact windows define synchronization triggers — the moments when agents may share beliefs and take joint actions.

1.1 Key Design Philosophy

The state space is built around **predicted miss distance at TCA** rather than raw RTN position. This choice is motivated by three factors:

1. Miss distance is what CDMs report and what operators reason about.
2. The collision reward depends directly on miss distance at TCA — using it as state makes the reward trivially correct.
3. It collapses a 6D relative state into a scalar, keeping the discrete model tractable.

Orbital propagation (Brahe) is used *offline only* to build the transition matrix. RS-SDA* operates entirely on precomputed T , O , R arrays.

Note on Communication Variables in State

The abstract describes a state space that includes communication variables (ground-station visibility, most recent comms status). In our implementation, these are **not** included as state variables. Instead, communication structure is encoded entirely via the **sync_states** list passed to `SDecPOMDPModel` — the set of states at which agents synchronize.

This is the standard pattern across all RS-SDA* benchmarks (Mars rovers, Tiger, Labyrinth): the physical state is the domain state only, and the trigger list encodes when coordination occurs. For our domain, ground contact windows are deterministic and known upfront, so the sync trigger list captures the communication structure completely without redundancy. Comms variables in state would add complexity with no expressive benefit given a fixed, known contact schedule.

2 Conjunction Geometry

2.1 Geometry Type: Radial-Miss

We use a **radial-miss** conjunction geometry. In the RTN (Radial-Transverse-Normal) frame centered on SC1:

- **R (Radial)**: points outward from Earth’s center through SC1. This is the *collision axis* — SC2 is offset in R at TCA.
- **T (Transverse / Along-track)**: points in the direction of orbital motion. SC2 approaches SC1 from behind (negative T), closing at $v_{\text{rel}} \approx 15 \text{ m/s}$.
- **N (Normal / Cross-track)**: perpendicular to the orbital plane. $\delta N \approx 0$ throughout — negligible cross-track motion.

At TCA, the geometry is:

$$\delta \mathbf{r}_{\text{RTN}}^{\text{TCA}} = \begin{bmatrix} d_{\text{miss}} \\ 0 \\ 0 \end{bmatrix}, \quad \delta \mathbf{v}_{\text{RTN}}^{\text{TCA}} = \begin{bmatrix} 0 \\ -v_{\text{rel}} \\ 0 \end{bmatrix} \quad (1)$$

where d_{miss} is the miss distance (radial separation at closest approach) and $v_{\text{rel}} = 15 \text{ m/s}$ is the along-track closing speed.

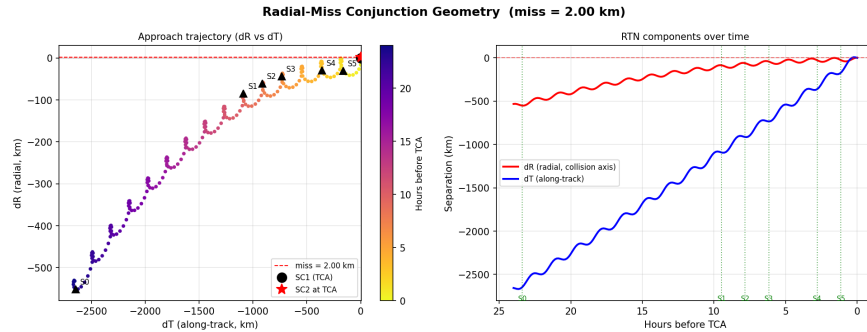


Figure 1: Radial-miss conjunction geometry in the RTN frame at TCA. SC2 is offset from SC1 in the radial direction by the miss distance. SC2 approaches from the along-track direction (negative T), with relative velocity entirely in T at TCA.

Why radial-miss (not crossing): A crossing conjunction has the miss distance in the N (cross-track) direction. Discretizing N into bins while keeping the RTN state tractable collapses the collision zone to a single bin that is never reached from nominal trajectories. Radial-miss places the collision axis in R, which is directly captured by the POMDP state (miss distance bin), and along-track maneuvers (Δv_T) shift the radial offset at TCA in a predictable, bin-differentiating way.

2.2 Orbit and Scenario Parameters

- **Orbit:** 550 km circular, 55° inclination
- **Planning horizon:** 24 h before TCA
- **Relative speed at TCA:** $v_{\text{rel}} = 15 \text{ m/s}$ (along-track)
- **Maneuver magnitude:** $\Delta v = 0.5 \text{ m/s}$ (configurable; produces good bin differentiation at all planning horizons)

2.3 Representative State for Matrix Generation

Since the POMDP state is a scalar miss distance bin, each state must be mapped to a representative 6D relative state for Brahe propagation. We use **back-propagation from TCA**: SC2 is placed at the target miss distance at TCA, then Brahe-propagated backward to the stage epoch.

1. Place SC2 at TCA with RTN relative state:

$$\delta \mathbf{r}_{\text{RTN}}^{\text{TCA}} = [d_{\text{miss}}, 0, 0]^\top, \quad \delta \mathbf{v}_{\text{RTN}}^{\text{TCA}} = [0, -v_{\text{rel}}, 0]^\top \quad (2)$$

2. Back-propagate SC2 from τ_{TCA} to τ_k (stage epoch) using Brahe.
3. Forward-propagate from τ_k to τ_{TCA} (round-trip check) — miss distance is recovered to $< 1 \text{ m}$ accuracy across all stages and all bins.

Why back-propagation: Placing SC2 at a radial offset at the *stage* epoch does not produce the target miss distance at TCA — the radial offset evolves under Keplerian dynamics and can differ by thousands of km after 24 h of propagation. Back-propagation from TCA guarantees the representative state has exactly the intended miss distance, independent of the planning stage.

3 SDec-POMDP Formulation

3.1 State Space

The global state at decision epoch k is:

$$s = (d_{\text{miss}}, k) \quad (3)$$

where d_{miss} is the predicted miss distance at TCA (meters), discretized into N_d bins, and $k \in \{0, \dots, H-1\}$ is the planning stage index.

Total states: $N_d \times H + 1 = 6 \times 6 + 1 = \mathbf{37}$ (including one sink state after the final stage).

Physical Interpretation of the State

The state (d_{miss}, k) is best understood as: *at the current contact window (stage k), what miss distance will the spacecraft have at TCA if no further maneuvers are applied?*

- **Stage 0 is $T-23.4 \text{ h}$** — the earliest ground contact, 23.4 hours before TCA. The initial state is the predicted miss distance at this epoch.

Table 1: Miss distance discretization

Bin	Range	Interpretation
0	[0, 1) km	Collision zone
1	[1, 5) km	High risk
2	[5, 20) km	Moderate risk
3	[20, 100) km	Low risk
4	[100, 500) km	Nominal approach
5	[500, ∞) km	Safe

- **WAIT invariant:** under no maneuver, the predicted miss distance at TCA does not change between stages (Keplerian dynamics are deterministic). A spacecraft in bin 0 at stage 0 will remain in bin 0 at TCA unless a burn is applied.
- **Starting in bin 0** means: at $T-23.4$ h, the current conjunction assessment predicts a close approach of < 1 km. This is a *pre-identified* conjunction scenario, not a collision in progress — the operators have 23 h to act.
- **Burning at stage k** shifts the predicted miss distance for all future stages. A prograde or retrograde burn of $\Delta v_T = 0.5$ m/s at stage 0 moves the prediction from bin 0 (~ 0.5 km) to bin 4 (~ 128 km).

Planning horizon and contact schedule:

Stage	Time before TCA	Interpretation
0	$T-23.4$ h	Initial assessment; first opportunity to act
1	$T-9.5$ h	Re-estimate miss from updated tracking
2	$T-7.8$ h	Update
3	$T-6.2$ h	Update
4	$T-2.8$ h	Last practical burn window
5	$T-1.1$ h	Final decision (small effect at late stage)
TCA	$T-0$ h	Closest approach; collision check; reward fires

3.2 Action Space

Each agent independently selects an impulsive along-track maneuver:

$$\mathcal{A}_i = \{\text{WAIT}, +\Delta v_T, -\Delta v_T\}, \quad i \in \{1, 2\} \quad (4)$$

Joint action space: $|\mathcal{A}| = 3^2 = 9$.

Δv_T is a fixed magnitude burn (e.g. 1 m/s for the discrete model; operationally realistic values are 2–50 mm/s). The burn is applied in the along-track direction in ECI coordinates.

3.3 Transition Model

The transition $T(s' | s, a)$ is built offline using Brahe:

1. From state $(d_{\text{miss_bin}}, k)$, reconstruct a representative RTN relative state at stage epoch τ_k .

2. Apply joint maneuver $a = (a_1, a_2)$ impulsively to SC1 and SC2.
3. Propagate both spacecraft from τ_k to TCA using Brahe numerical integrator (two-body or with perturbations).
4. Compute resulting miss distance d' and bin it $\rightarrow$ next state $(d'_{\text{bin}}, k + 1)$.
5. $T[a, s, s'] = 1$ (deterministic given bin-center representative state).

The final stage ($k = H - 1$) transitions to the sink state.

Key property: Because the state already encodes predicted miss distance, the collision reward fires cleanly at the terminal transition without any additional propagation.

3.4 Observation Model

Each agent i receives a noisy estimate of the miss distance during ground contact, modeled as Gaussian noise on the true miss distance:

$$\tilde{d}_i \sim \mathcal{N}(d_{\text{miss}}, \sigma_{\text{obs}}^2) \quad (5)$$

with $\sigma_{\text{obs}} \approx 10\text{--}50$ km (realistic CDM uncertainty).

Observations are discretized into the same N_d bins. The observation probability is:

$$O(o \mid s, a) = \int_{\text{bin}_o} \mathcal{N}(\tilde{d}; d_{\text{miss}}(s'), \sigma_{\text{obs}}^2) d\tilde{d} \quad (6)$$

Joint observation: $o = (o_1, o_2)$, $|\mathcal{O}| = N_d^2 = 36$.

Between ground contacts: agents receive no observation update and act on stale beliefs. This is the core source of partial observability in the model.

3.5 Reward Function

$$R(s, a) = R_{\text{collision}}(s) + R_{\text{maneuver}}(a) + R_{\text{comms}}(a) \quad (7)$$

- **Collision penalty** (terminal, at last stage):

$$R_{\text{collision}}(s) = \begin{cases} -1000 & \text{if } d_{\text{miss}} < 1 \text{ km at TCA} \\ 0 & \text{otherwise} \end{cases} \quad (8)$$

- **Maneuver cost** (fuel / trajectory deviation proxy):

$$R_{\text{maneuver}}(a) = -10 \cdot \mathbf{1}[a_1 \neq \text{WAIT}] - 10 \cdot \mathbf{1}[a_2 \neq \text{WAIT}] \quad (9)$$

- **Communication cost** (Scenario 3 only):

$$R_{\text{comms}}(a) = -c_{\text{sync}} \cdot \mathbf{1}[\text{sync event at this stage}] \quad (10)$$

[TODO: Add trajectory deviation reward: penalize change in orbital elements (e.g. semi-major axis deviation) accumulated over all maneuvers. This is the “mission performance” term from the abstract.]

Table 2: Decision epochs for Scenario 1 (single GS)

Stage k	Time before TCA	Epoch (UTC)
0	−23.39 h	2025-06-01 00:36
1	−9.48 h	2025-06-01 14:31
2	−7.83 h	2025-06-01 16:10
3	−6.17 h	2025-06-01 17:49
4	−2.80 h	2025-06-01 21:12
5	−1.13 h	2025-06-01 22:52

3.6 Decision Epochs and Planning Horizon

Decision epochs are the midpoints of ground-station contact windows, computed via Brahe’s `location_accesses()` visibility analysis (`examples/test_contacts.py`). For a 550 km, 55° orbit over Goddard Space Flight Center (lon = -76.8° , lat = 39.0° , 10° min elevation), Brahe finds 6 SC1 passes over the 24 h planning horizon. The nominal 6-contact schedule is:

Note on the contact gap: There is a ~ 14 h gap between Stage 0 ($T-23.4$ h) and Stage 1 ($T-9.5$ h). This is a real orbital mechanics feature: at 55° inclination over a mid-latitude station, the spacecraft is below the 10° elevation horizon for roughly half its orbital period (~ 95 min orbit). Stages 1–5 cluster in the final 10 h before TCA because the spacecraft makes several successive passes during that window. This gives the policy many update opportunities late in the horizon but a long “blind” period early on.

For Scenario 1, both spacecraft are in nearly identical orbits, so all 6 SC1 passes are simultaneously visible to SC2 $\Rightarrow$ all 6 contact windows are joint $\Rightarrow$ all 36 non-sink states are synchronization triggers.

3.7 Synchronization Triggers

Synchronization triggers are defined by ground-station contact windows. A state is a sync trigger if both spacecraft have an active (or recent) contact at that stage, enabling joint belief update and coordinated action.

- **Scenario 1** (cooperative): all 6 stages are sync triggers $\rightarrow$ always centralized
- **Scenario 2** (asymmetric): SC2 follows a fixed policy; SC1 syncs at stages 0–2 only
- **Scenario 3** (costly comms): all stages available but sync incurs R_{comms} penalty; policy trades off coordination vs. cost

4 System Architecture

The system has three distinct components that run at different times:

4.1 Component 1: Offline Matrix Builder (`spacecraft_matrices.py`)

Runs once. Uses Brahe to precompute T , O , R matrices and saves to `.npz` cache.

For each (`miss_bin`, `stage`, `joint_action`):

1. Reconstruct RTN relative state at stage epoch

2. Apply maneuvers to SC1 and SC2
3. Brahe-propagate both to TCA
4. Measure miss distance $\rightarrow$ bin $\rightarrow$ $T[a, s, s']$
5. Compute observation distribution $\rightarrow$ $O[a, s', o]$
6. Assign reward $\rightarrow$ $R[a, s]$

Save $T, O, R, \text{init_b}$ to `spacecraft_matrices_cache.npz`

4.2 Component 2: Policy Solver (`sdec_spacecraft.py`)

Loads cached matrices, constructs `SDecPOMDPModel`, runs RS-SDA*. Never calls Brahe. Outputs a policy $\pi(\text{belief}) \rightarrow \text{action}$.

```
T, O, R, init_b = load_matrices()
model = SDecPOMDPModel(nagents=2, ...)
config = RSSDAConfig(maxh=6, algorithm="approximate", ...)
sdec = SDecPOMDP(model, config)
value, policy = sdec.multi_agent_astar(6)
```

4.3 Component 3: Simulator (`spacecraft_simulator.py`)

Runs after policy is computed. Uses Brahe with real ECI states to evaluate the policy in closed-loop simulation. Generates paper metrics.

Initialize true ECI states from conjunction generator
 For each stage k :

1. Propagate true states to stage epoch with Brahe
2. Simulate noisy observation (miss distance estimate)
3. Update belief
4. If sync trigger: share beliefs, take joint action
5. Else: each agent acts independently per policy
6. Apply maneuver to true ECI states

At TCA: propagate to TCA, measure true miss distance
 Record: miss distance, total dv, sync count, trajectory deviation

4.4 Data Flow Diagram

```
OFFLINE (once, ~minutes)
=====
Brahe (orbit propagation)
Brahe (GS contact windows)
    |
    v
spacecraft_matrices.py --> T, O, R, init_b --> .npz cache
```

```
POLICY COMPUTATION (once, ~seconds to minutes)
=====
.npz cache
    |
```

```

      v
sdec_spacecraft.py --> RS-SDA* --> policy pi(belief) -> action

SIMULATION (per scenario run)
=====
Conjunction generator (Brahe)
    |
    v
spacecraft_simulator.py --> closed-loop rollout with policy
    |
    v
Metrics: miss distance, dv total, sync count, trajectory deviation

```

5 Three Scenarios

5.1 Scenario 1: Cooperative Shared Operator

Both spacecraft controlled by one operator with access to a single ground station (Goddard). All 6 contacts are joint $\rightarrow$ always synchronized.

Experiment: vary ground network density.

- Dense (6–12 GS globally): many sync triggers, near-centralized
- Regional (1–3 GS): fewer triggers, more decentralized intervals

Baseline comparisons:

- Centralized: perfect information at all times
- Decentralized: no synchronization ever
- SDec: ground-contact sync triggers

5.2 Scenario 2: Asymmetric Control

SC1 is fully controllable. SC2 follows a fixed policy (third-party operator). SC1 must plan under uncertainty about SC2’s behavior.

SC2 policy variants:

- *Uncooperative*: SC2 never maneuvers
- *Reactive*: SC2 maneuvers only if predicted miss $< \theta$ at its next contact, where threshold θ is unknown to SC1

SC1’s observation model must incorporate uncertainty about whether SC2 has maneuvered, widening its effective miss-distance uncertainty.

[**TODO: Define how SC2’s fixed policy is encoded in the POMDP transition matrix.**
Options: (a) marginalize over SC2 actions with prior probabilities, (b) expand state to include SC2 behavior type as a hidden variable.]

5.3 Scenario 3: Costly Communication

Same contact structure as Scenario 1, but each synchronization event incurs a penalty c_{sync} in the reward. Policy must trade off:

- Sync $\rightarrow$ better belief, coordinated action, but costs c_{sync}
- No sync $\rightarrow$ free, but agents act on independent beliefs

Experiment: sweep c_{sync} from 0 to 100, measure optimal sync count and resulting miss distance.

6 Evaluation Metrics

From the abstract:

1. **Minimum miss distance** at TCA (meters) — primary safety metric
2. **Cumulative trajectory deviation** — **[TODO: define: total Δv accumulated, or change in semi-major axis, or tracking error norm]**
3. **Total synchronization events** — coordination overhead
4. **Sync timing** relative to ground windows and conjunction geometry

Policy comparison:

- V_{cen} : centralized POMDP value (perfect info upper bound)
- V_{dec} : fully decentralized value (no sync lower bound)
- V_{sdec} : semi-decentralized value (our approach)
- Key result: $V_{\text{dec}} \leq V_{\text{sdec}} \leq V_{\text{cen}}$, with $V_{\text{sdec}} \approx V_{\text{cen}}$ using few syncs

7 Implementation Status

Scenario 1 Results (100 rollouts, $dv = 0.5 \text{ m/s}$) — After bin-1 fix

Notes:

- Bin 0 (collision zone): all variants maneuver exactly once at stage 0, reaching bin 4 ($\sim 128 \text{ km}$). Policy value = -10 (one maneuver cost).
- Bin 1 (1–5 km): now also maneuvers after adding the -200 terminal penalty. Policy correctly avoids lingering in the high-risk zone.
- Centralized = SDec: expected for Scenario 1 (all stages sync).
- Decentralized achieves identical outcomes: with deterministic dynamics and informative observations ($\sigma = 5 \text{ km}$), independent agents reach the same decision without coordination. This highlights a current limitation — the problem is not yet hard enough to differentiate the three variants.

Table 3: Component implementation status

Component	File	Status
GS contact windows	<code>examples/test_contacts.py</code>	Done
Conjunction geometry	<code>examples/test_conjunction_geometry.py</code>	Done
Miss-distance discretizer	<code>spacecraft_discretizer.py</code>	Done
Matrix builder	<code>spacecraft_matrices.py</code>	Done
Policy solver	<code>sdec_spacecraft.py</code>	Done
Simulator	<code>spacecraft_simulator.py</code>	Done (RS-SDA* + greedy)
Scenario 1 comparison	<code>spacecraft_simulator.py --compare</code>	Done (100 rollouts)
Scenario 3 sweep script	<code>scenario3_sweep.py</code>	Done (run at $c_{\text{sync}} \in [0, 200]$)
Policy grid visualization	<code>plot_policy.py</code>	Done
Bin-1 terminal penalty	<code>spacecraft_matrices.py</code>	Done ($R = -200$ at stage 5)
Observation noise tuning	<code>spacecraft_matrices.py</code>	Done ($\sigma = 5$ km)

Table 4: Scenario 1: Cooperative shared operator. All 6 stages are joint sync triggers. Bin-1 terminal penalty = -200 applied. Observation noise $\sigma = 5$ km.

Policy	Init bin	Coll.%	Mean miss (km)	Mean dv (m/s)	Mean syncs
Centralized	0	0.0	127.75	0.50	6.0
Decentralized	0	0.0	128.01	0.50	0.0
SDec	0	0.0	127.75	0.50	6.0
Centralized	1	0.0	127.90	0.50	6.0
Decentralized	1	0.0	128.09	0.50	0.0
SDec	1	0.0	127.90	0.50	6.0

- **Open design issue:** transitions are deterministic (bin-center representative state). Observation noise ($\sigma = 5$ km) is the only source of uncertainty. This makes the POMDP easier than a real conjunction scenario. Future work: stochastic T via sampling within bins, finer discretization in danger zone.

Scenario 3 Sweep Results ($c_{\text{sync}} \in [0, 200]$, `init_bin=0`)

Table 5: Scenario 3: Costly communication sweep. Sync count fixed at 6 regardless of cost — indicates a structural modeling issue (sync is mandatory, not optional in current formulation).

c_{sync}	Policy val	Coll.%	Mean miss (km)	Mean dv (m/s)	Mean syncs
0	-10	0.0	127.75	0.50	6.0
20	-130	0.0	127.75	0.50	6.0
60	-370	0.0	127.75	0.50	6.0
100	-610	0.0	127.75	0.50	6.0
200	-1210	0.0	127.75	0.50	6.0

Root cause: All 36 non-sink states are `sync_states`, making synchronization structurally mandatory whenever the belief has mass in any non-sink state. The reward penalty accumulates (policy

value degrades linearly) but cannot alter the sync/dec decision. Fix: model sync as optional via `sync_actions` or `sync_observations` in the RSSDA API, so the policy can choose to skip a contact window when c_{sync} is high.

Open tasks and future work:

1. **Scenario 3 structural fix:** make sync optional — use `sync_actions` or encode a “communicate” vs. “don’t communicate” choice during contact windows.
2. **Scenario 2:** encode SC2’s fixed (uncooperative) policy in the transition matrix: $T_{\text{asym}}[a, s, s'] = T_{\text{full}}[(a_1 \cdot 3 + 0), s, s']$.
3. **Reward shaping:** replace flat $-1000/-200$ thresholds with a continuous function of miss distance, e.g. $R_{\text{collision}}(d) = -1000 \cdot \exp(-d/d_{\text{safe}})$.
4. **Trajectory deviation cost:** penalize total Δv accumulated or change in semi-major axis to model fuel cost more accurately.
5. **Finer bins:** split bin 0 into $[0, 0.1)$ and $[0.1, 1)$ km; split bin 1 into $[1, 2)$ and $[2, 5)$ km — 8 bins total.
6. **Observation noise sweep:** evaluate centralized vs. SDec vs. decentralized performance across $\sigma \in \{1, 5, 20\}$ km.
7. **Multiple dv magnitudes:** sweep $dv \in \{0.1, 0.5, 1.0\}$ m/s.
8. **Policy visualization:** `plot_policy.py` generates a (miss_bin $\times$ stage) action grid — extend to show belief-weighted policies and compare centralized vs. decentralized side-by-side.